🚀 TPC WEBSITE – COMPLETE STEP-BY-STEP PROCEDURE

We'll divide it into 5 Phases.

You follow this strictly. No jumping.

🟢 PHASE 1: PROJECT FOUNDATION (Day 1–3)

✅ Step 1: Create Project Structure

Create two folders:

```
tpc-system/
  ├── frontend/
  └── backend/
```

✅ Step 2: Initialize Backend

Inside backend:

```
npm init -y
npm install express pg dotenv cors bcrypt jsonwebtoken
npm install prisma --save-dev
npm install @prisma/client
```

Initialize Prisma:

```
npx prisma init
```

✅ Step 3: Setup PostgreSQL

- Install PostgreSQL (if local)
- Create database:

```
CREATE DATABASE tpc_db;
```

- Add DATABASE_URL in .env

✅ Step 4: Design Database Schema FIRST

Inside schema.prisma create models:

- User
- StudentProfile
- Company
- JobPost

- Application
- Interview

Then run:

npx prisma migrate dev --name init

👉 IMPORTANT: Don’t write backend routes before DB is ready.

🟢 PHASE 2: BACKEND CORE (Day 4–10)

Now backend logic.

✅ Step 5: Create Folder Structure

Inside backend:

```
src/
  ├── routes/
  ├── controllers/
  ├── middleware/
  ├── services/
  ├── utils/
  └── app.js
```

✅ Step 6: Setup Express Server

- Basic server
- CORS
- JSON parsing
- Error middleware

Test with Postman first.

✅ Step 7: Authentication System

Create:

- Register API
- Login API
- Password hashing (bcrypt)
- JWT token generation

Then create:

- authMiddleware
- roleMiddleware

Test:

- Student login
- Admin login
- Company login

This is CRITICAL.

✅ **Step 8: Build Core APIs**

Build in this order:

1️⃣ Create Student Profile
2️⃣ Create Company
3️⃣ Post Job
4️⃣ View Jobs
5️⃣ Apply Job
6️⃣ Update Application Status
7️⃣ Schedule Interview

Test each API in Postman before moving.

🟢 **PHASE 3: FRONTEND START (Day 11–18)**

Now shift to frontend.

✅ **Step 9: Initialize React**

Inside frontend:

npx create-react-app .
npm install axios react-router-dom

✅ **Step 10: Create Basic Pages**

- Login
- Register
- Student Dashboard
- Admin Dashboard
- Company Dashboard

- Job Listing Page

✅ Step 11: Connect Backend

Use Axios:

- Store JWT in localStorage
- Send token in headers
- Protect routes

Create:

- ProtectedRoute component

🟢 PHASE 4: REAL FEATURES (Day 19–25)

Now make it professional.

✅ Step 12: Resume Upload

- Install Multer
- Store file locally or Cloudinary

✅ Step 13: Filtering & Pagination

- Filter jobs by branch
- Filter by salary
- Add pagination

✅ Step 14: Dashboard Analytics

Install Chart.js

Show:

- Total jobs
- Total students
- Placement %
- Average salary

This makes project look industry-level.

🟢 PHASE 5: POLISH & DEPLOY (Day 26–30)

Now make it production ready.

✅ Step 15: Add Security

Install:

npm install helmet morgan express-rate-limit

Add:

- Rate limiting
- Logging
- Secure headers

✅ Step 16: Clean Code

- Remove console logs
- Add proper error messages
- Use async/await properly
- Create reusable functions

✅ Step 17: Deployment

Backend → Render
Database → Neon PostgreSQL
Frontend → Vercel

Test live.